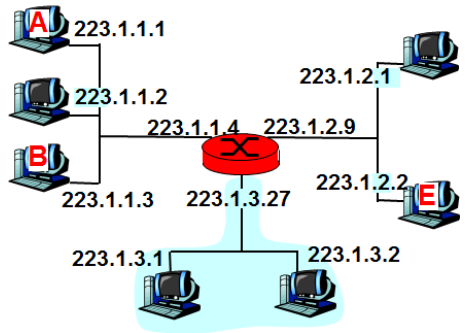
Redes de Datos II
HTTP/2

Matías Robles

LINTI–UNLP

28 de octubre de 2026



BY NC SA

Contenidos

1. HTTP/2 - Introducción
2. Funcionamiento
3. Stream y Frames
4. Control de Flujo
5. Server Push
6. Header Compression

Estamos en:

1 HTTP/2 - Introducción

2 Funcionamiento

3 Stream y Frames

4 Control de Flujo

5 Server Push

6 Header Compression

Problemas en HTTP/1.1

- ¿Por qué necesitamos irnos de HTTP/1.1?
- HTTP/1.1 lleva más de 20 años de vida (1997).
- Es un protocolo sincrónico (envía solicitud y se bloquea esperarando la respuesta) basado en texto
- Soporta pipelining, pero no fue implementado o habilitado. Head-of-line blocking
- *Trucos* para mejorar su performance:
  - Usar múltiples conexiones HTTP.
  - Realizar solicitudes más grandes.

Historia

- 2009: Belsh and Peon, en Google, proponen SPDY como alternativa a HTTP.
- 2012: HTTP Working Group es redefinido para trabajar en una nueva versión de HTTP.
- Diciembre 2014: HTTP WG presenta la *Proposed Standard* a la Internet Engineering Steering Group (IESG).
- Mayo 2015: RFC 7540. Hypertext Transfer Protocol Version 2 (HTTP/2).
- Finales de 2015: mayoría de los browser ya soportan HTTP/2.
- Junio 2022: RFC 9113. HTTP/2 (Obsoleta RFC 7540).

Metas propuestas

- Mejorar sustancialmente la latencia percibida por el usuario final al usar HTTP/1.1 sobre TCP.
- Solucionar el problema *head of line blocking* en HTTP.
- No requerir múltiples conexiones a un servidor para habilitar paralelismo.
- Mantener la semántica de HTTP/1.1, incluyendo métodos HTTP, código de estado, URIs y los campos de cabecera.
- Definir de manera clara cómo HTTP/2 interactúa con HTTP/1.x.
- Identificar de manera clara nuevas extensiones y políticas de uso.

Nuevas características

- Protocolo binario. No basado en texto.
- Multiplexado en lugar de sincrónico.
- Control de flujo.
- Priorización de streams (deprecated).
- Compresión de cabeceras.
- Server push.

Descripción General

- Protocolo de la capa de aplicación orientado a conexión. Extensible.
- Ejecuta sobre TCP (cliente es el iniciador de la conexión TCP).
- Conexiones son persistentes.
- Cliente no debería abrir más de una conexión TCP al servidor.
- En HTTPS debe usarse TLS 1.2 o mayor (TLS debe soportar la extensión Server Name Indication(SNI)).
- **Stream**: flujo bidireccional de frames dentro de una conexión.
- **Frame:** es la unidad básica del protocolo. Cada tipo de frame sirve un propósito diferente.
- Cada frame pertenece a un *stream* en particular.
- No compatible con otras versiones de HTTP.

Estamos en:

1. HTTP/2 - Introducción
2. Funcionamiento
3. Stream y Frames
4. Control de Flujo
5. Server Push
6. Header Compression

Funcionamiento

- HTTP/2 utiliza los mismos esquemas URI, HTTP/HTTPS, y los mismos puertos por defecto, 80 y 443, que HTTP/1.1.
- Una implementación debe descubrir si el otro extremo soporta HTTP/2.
- Distintos procedimientos para HTTP y HTTPS.
- Existen dos identificadores:.
  - **h2c:** usado en el mecanismo HTTP Upgrade. Obsoleto.
  - **h2:** HTTP/2 utiliza Transport Layer Security (TLS).
- Cliente hace una solicitud HTTPS usando TLS con la extensión ALPN (Application-Layer Protocol Negotiation).
- Una vez finalizada la negociación TLS, cliente y servidor deben enviar el *prefacio de conexión*.
- Para HTTP, el cliente aprendió, por otro medio, que el servidor soporta HTTP/2 y envía directamente el prefacio de conexión.

Prefacio de conexión

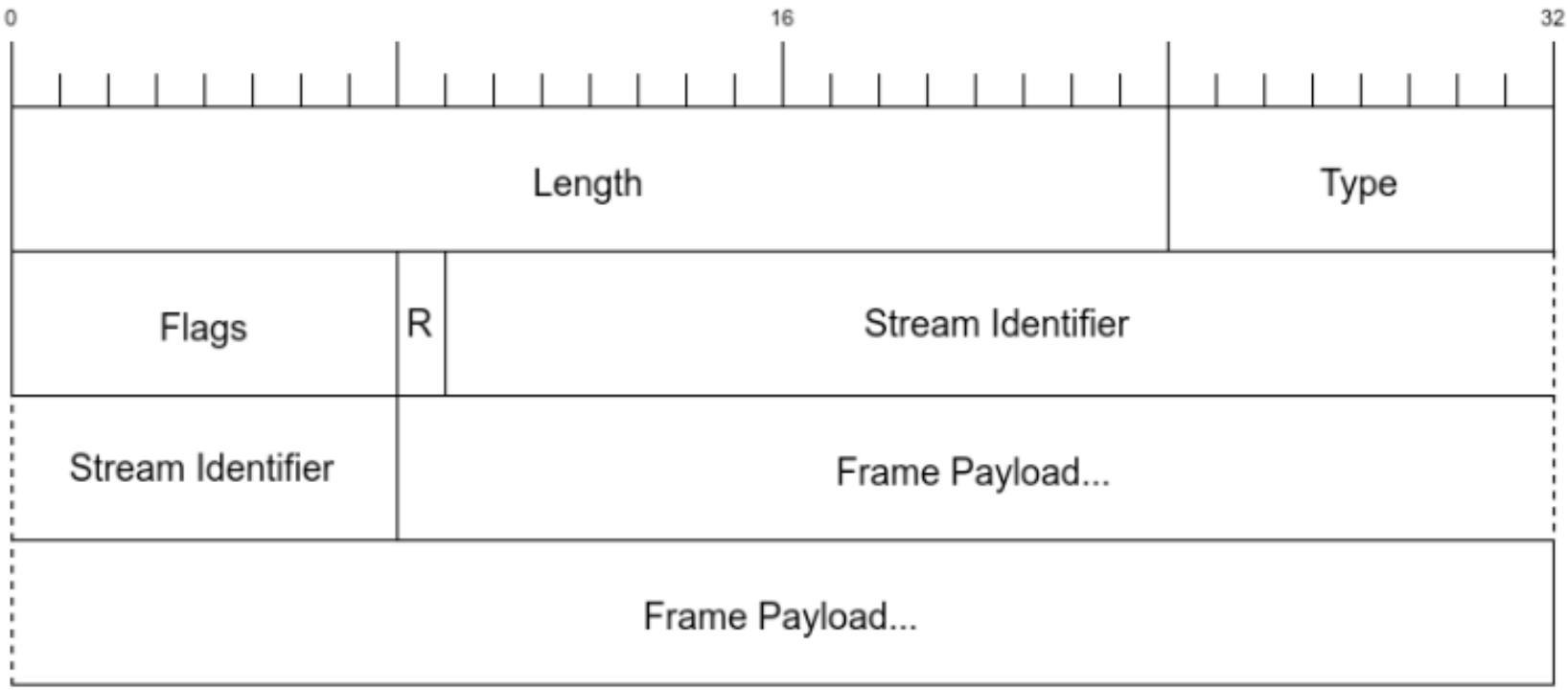
- Cada extremo debe enviar un prefacio de conexión con el fin de confirmar el uso del protocolo.
- Cliente y servidor envían diferente prefacio.
- Su finalidad es causar un error explícito en el otro extremo si no habla HTTP/2.
- Prefacio del cliente comienza con una secuencia hexadecimal de 24 octetos:
  - **Hexa:** 0x505249202a20485454502f322e300d0a0d0a534d0d0a0d0a
  - **ASCII:** PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n
- Cliente envía inmediatamente un frame *SETTINGS*, posiblemente vacío.
- Prefacio de conexión del servidor consiste de un frame *SETTINGS*, que es el primer frame que envía el servidor.
- Cada frame recibido como parte del prefacio de conexión debe ser reconocido (**ACK).**

Estamos en:

1. HTTP/2 - Introducción
2. Funcionamiento
3. Stream y Frames
4. Control de Flujo
5. Server Push
6. Header Compression

Frames

- Extremos empiezan a intercambiar *frames* una vez la conexión HTTP/2 fue establecida.
- Unidad más pequeña que transporta datos (cabeceras HTTP, payload, etc).
- Todos los frames comienzan con una cabecera fija de 9 bytes:



Frames . . .

- Entre los campos de la cabecera del frame:
  - **Length:** entero de 24 bits, sin signo, expresado en octetos. Indica la longitud del payload (no incluye a esta cabecera).
  - **Type:** indica qué tipo de frame es.
  - **Flags:** valores booleanos específicos al tipo de frame.
  - **Stream Identifier:** entero, de 31 bits, sin signo. Indica a qué stream pertenece. Valor 0 reservado para los frames asociados con la conexión como un todo.
- Toda implementación debe poder procesar payloads de longitud variable de hasta 2^{14} (16834) bytes.
- Payloads mayores, hasta 2^{31} - 1, deben ser indicados por el receptor (SETTINGS_MAX_FRAME_SIZE).
- Distintos tipos de frames: HEADERS, DATA, SETTINGS, PUSH_PROMISES, RST_STREAM, PING, GOAWAY, WINDOWS_UPDATE y CONTINUATION.

Tipos de frames

- **HEADERS**:
  - Tipo 0x01. Usado para abrir un stream.
  - No se envían en el stream con identificación 0.
  - Transporta cabeceras del requerimiento y/o respuesta.
  - Si las cabeceras no entran un solo frame, se envían en frames *CONTINUATION*.
- **DATA**:
  - Tipo 0x00. Transporta bytes de longitud variable asociados a un stream.
  - Flag *END_STREAM* indica que es el último frame enviado para ese stream.
  - No se envían en el stream con identificación 0.
- **RST_STREAM**:
  - Tipo 0x03. Permite el cierre inmediato de un stream.
  - Emisor no envía nuevos frames, pero si podría recibir.
  - No se envían en el stream con identificación 0.

Tipos de frames

- **SETTINGS**:
  - Tipo 0x04. Transporta parámetros de configuración que afectan a la conexión tales como preferencias y restricciones.
  - Enviados en el stream con identificador 0.
  - Debe ser enviado por cada extremo al inicio de la conexión.
  - También se usan para reconocer (ACK) la recepción de este tipo de frame.
  - Permite establecer distintos seteos: SETTINGS_MAX_CONCURRENT_STREAMS, SETTINGS_INITIAL_WINDOW_SIZE, SETTINGS_MAX_FRAME_SIZE, etc.
- **PUSH_PROMISE**:
  - Tipo 0x05. Solo pueden ser enviados por el servidor.
  - Permite avisar de streams que el emisor va a utilizar.
  - Servidor va a enviar objetos que el cliente no solicitó.
  - No es necesario crearlos en el mismo orden en el que se avisan.

Tipos de frames

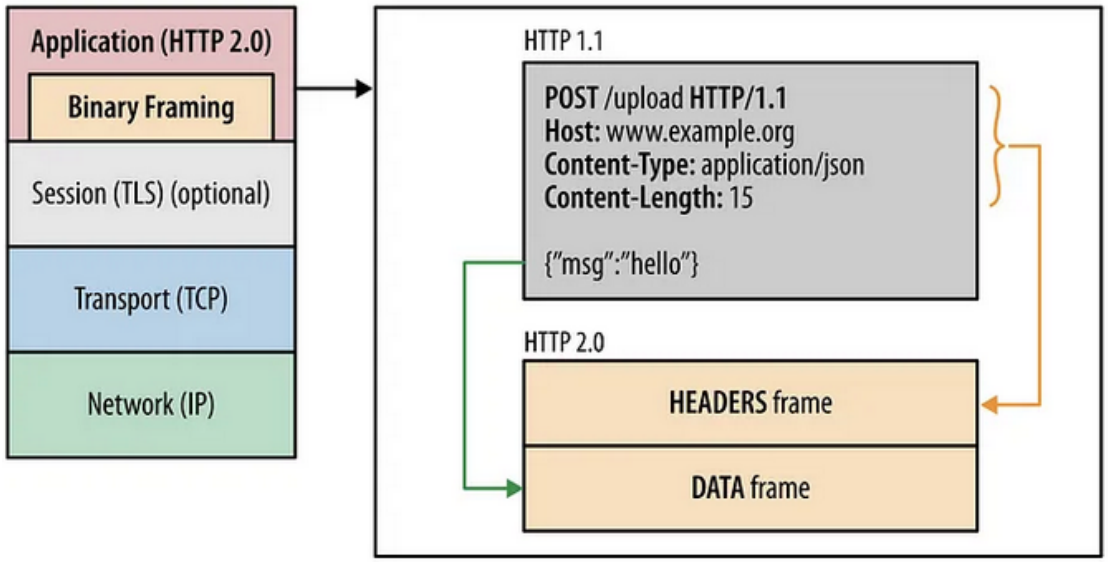
- **PING**:
  - Tipo 0x06. Mecanismo para medir el round-trip time y para determinar si una conexión aún está activa.
  - Si el flag ACK está seteado, frame PING es una respuesta.
  - Se envía con identificador de stream 0 y con un payload de 8 bytes.
- **GOAWAY**:
  - Tipo 0x07. Permite iniciar el cierre *elegante* de una conexión o indicar condiciones de error.
  - Usado a nivel de conexión, identificador de stream 0.
  - Emisor indica el último identificador de stream recibido y procesado.
  - No deben generarse nuevos streams.
- **WINDOWS_UPDATE**:
  - Tipo 0x08. Utilizado para el control de flujo en los streams o la conexión.
  - Indica cuantos bytes adicionales el emisor puede transmitir.
  - Solo se aplica a frames sujetos a control de flujo: *DATA*.

Streams

- Secuencia bidireccional de frames intercambiados entre el cliente y el servidor dentro de una conexión.
- En una conexión puede haber varios streams abiertos de manera concurrente.
- Cada stream se corresponde con una solicitud/respuesta dentro de una conexión.
- Cliente inicia un stream cada vez que realiza una nueva solicitud. El servidor responde en el mismo stream.
- Un stream puede ser cerrado por cualquiera de los extremos.
- Cada stream está identificado por un entero asignado por el extremo que lo inicia.
- Streams iniciados por el cliente debe seleccionar número impares como identificador. Servidores, pares.
- Es importante el orden en el que se envían los frames. Se procesan en el orden que se reciben.

Binary Framing Layer

- Determina cómo los mensajes HTTP son encapsulados y transmitidos entre el cliente y el servidor



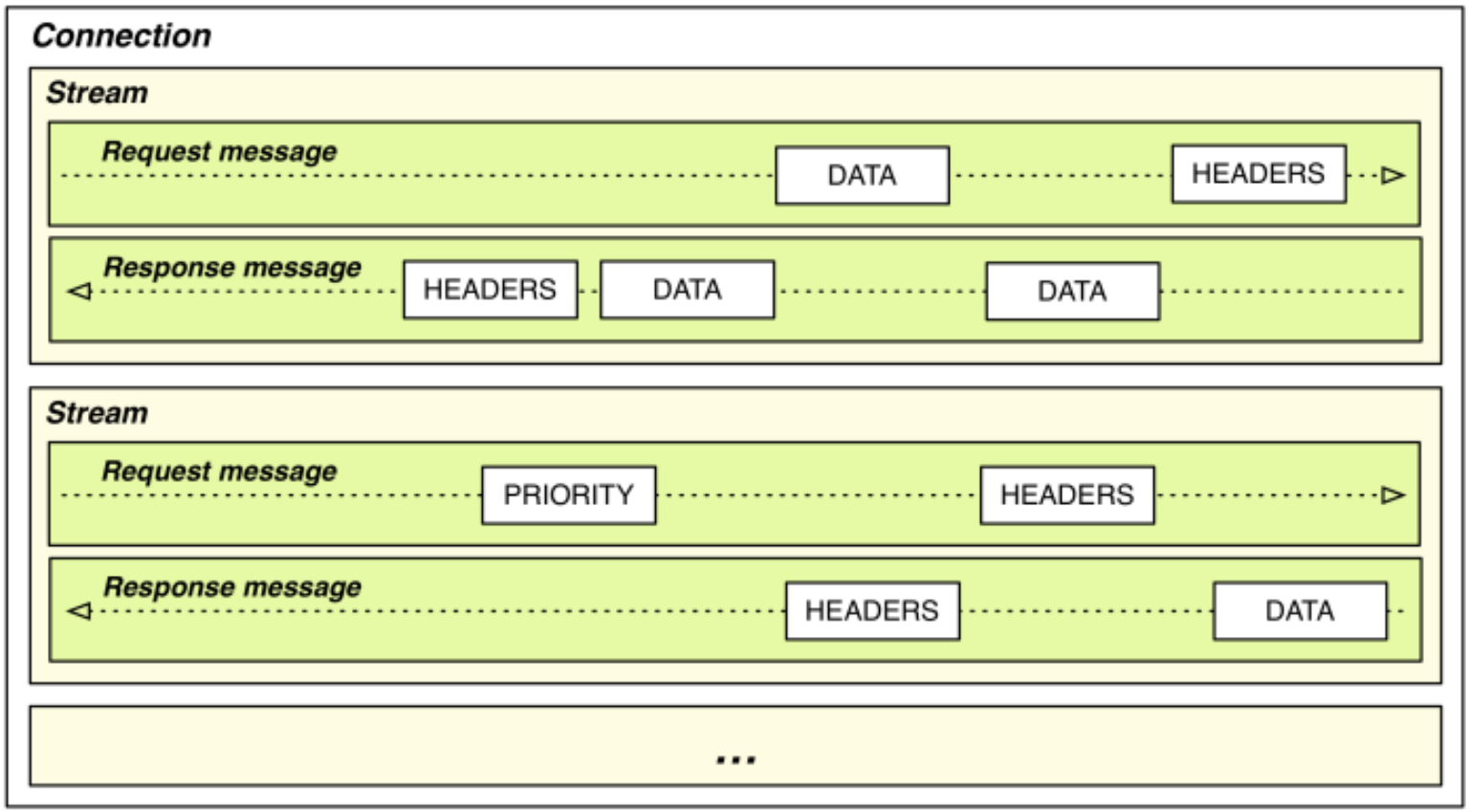
Fuente: Grigork Ilya (2015). HTTP/2: A New Excerpt from High Performance Browser Networking. O'Reilly.

Relación conexión, streams y frames

- Toda comunicación se ejecuta sobre una única conexión TCP.
- Cada conexión puede tener varios streams abiertos simultáneamente.
- Cada extremo puede limitar la cantidad de streams concurrentes que puede abrir el otro extremo.
- Stream con identificador 0 se usa para enviar mensajes de control de la conexión.
- Cada nuevo identificador debe ser numéricamente mayor a todos los elegidos previamente por ese extremo.
- Cada solicitud/respuesta es un stream consistirá de uno ó más frames.
- Frames pertenecientes a distintos streams se pueden intercalar.
- Un cliente envía una solicitud HTTP en un nuevo stream usando un identificador no usado previamente.
- Servidor envía la respuesta en el mismo stream que el de la solicitud.

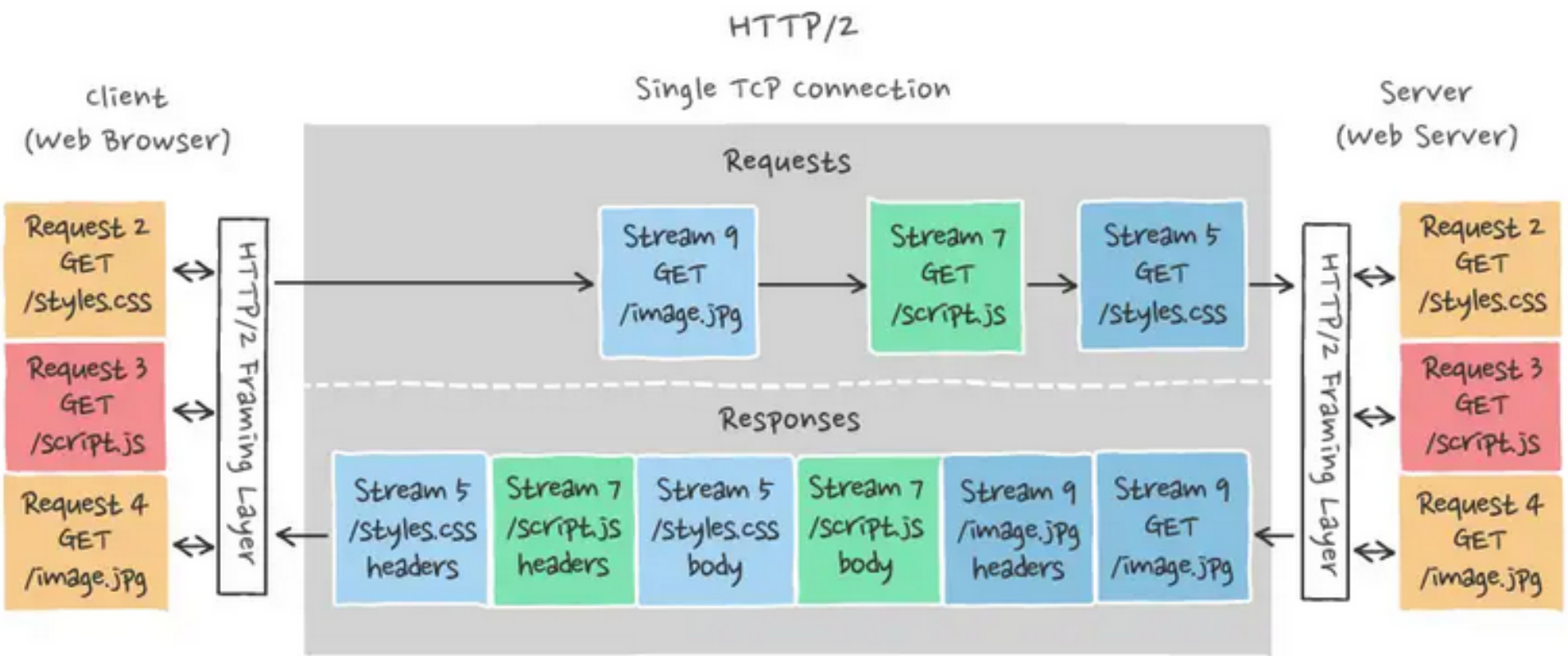
Relación conexión, streams y frames

- Una solicitud o respuesta HTTP/2 consiste de:
  - Un frame HEADER
  - Cero ó más frames DATA
  - Opcionalmente, un frame HEADER



Fuente: Grigork Ilya (2015). HTTP/2: A New Excerpt from High Performance Browser Networking. O'Reilly.

Relación conexión, streams y frames



Fuente: https://ably.com/topic/http2

HTTP/2 - Pseudo-Headers

- Todo es una cabecera.

```
GET / HTTP/1.1
Host: www.example.com
User-agent: Next-Great-h2-browser-1.0.0
Accept-Encoding: compress, gzip
```

```
:scheme: https
:method: GET
:path: /
:authority: www.example.com
User-agent: Next-Great-h2-browser-1.0.0
Accept-Encoding: compress, gzip
```

```
HTTP/1.1 200 OK
Content-type: text/plain
Content-length: 2
```

```
:status: 200
content-type: text/plain
```

- ¿Y la cabecera Content-Length?

Estamos en:

1. HTTP/2 - Introducción
2. Funcionamiento
3. Stream y Frames
4. Control de Flujo
5. Server Push
6. Header Compression

Control de Flujo

- Streams compiten entre sí por el uso de la conexión TCP.
- TCP no puede aplicar control de flujo sobre los streams individuales.
- Cada emisor mantiene una ventana por cada stream.
- Valor entero que indica cuantos bytes de datos el emisor puede transmitir: medida de la capacidad de *buffering* del receptor.
- Es unidireccional. Cada extremo anuncia el tamaño máximo de ventana para la conexión y cada stream.
- Frame *SETTINGS* permite modificar el tamaño de la ventana al inicio de la conexión.
- Tamaño por defecto: 65.535 bytes (máximo: 2^{31} - 1).
- Uso del frame dedicado para actualizar la ventana.
- Solo se aplica sobre los frames *DATA*.

Control de Flujo

- *WINDOW_UPDATE* puede ser específico para un stream o para la conexión entera (stream_identifier).
- Se mantiene una ventana por cada stream y la conexión como un todo.
- No se debe enviar datos con una longitud que exceda la ventana del stream y de la conexión.
- Emisor reduce el espacio disponible de ambas ventanas por la longitud del frame enviado.
- Receptor envía frames *WINDOW_UPDATE* para liberar espacio (frames separados).
- Emisor actualiza su ventana según la longitud recibida en el frame *WINDOW_UPDATE*.

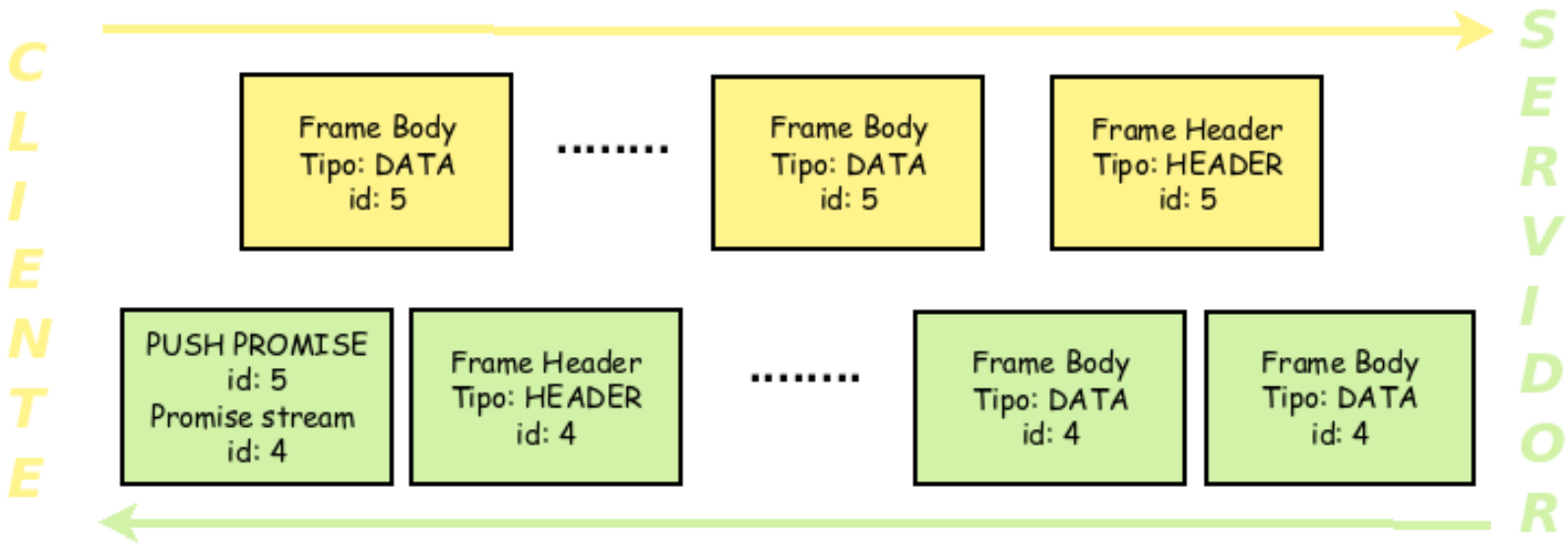
Estamos en:

1. HTTP/2 - Introducción
2. Funcionamiento
3. Stream y Frames
4. Control de Flujo
5. Server Push
6. Header Compression

Server Push

- Servidor puede enviar respuestas (objetos) antes que el cliente se los solicite.
- Servidor envía un frame *PUSH_PROMISE*.
- El identificador del stream es el mismo de la solicitud al que la respuesta está asociada.
- Frame contiene una bloque de cabeceras similar al de la solicitud del cliente.
- Objeto enviado debe ser cacheable y el método debe ser seguro (idempotente).
- *PUSH_PROMISE* indica cual es el identificador de stream que se usará para enviar la respuesta.
- Cliente puede resetear el nuevo stream con un *RST_STREAM* o *PROTOCOL_ERROR* en un frame *GOAWAY*.

Server Push


CLIENTE
SERVIDOR
Frame Body
Tipo: DATA
id: 5
.....
Frame Body
Tipo: DATA
id: 5
Frame Header
Tipo: HEADER
id: 5
PUSH PROMISE
id: 5
Promise stream
id: 4
Frame Header
Tipo: HEADER
id: 4
.....
Frame Body
Tipo: DATA
id: 4
Frame Body
Tipo: DATA
id: 4

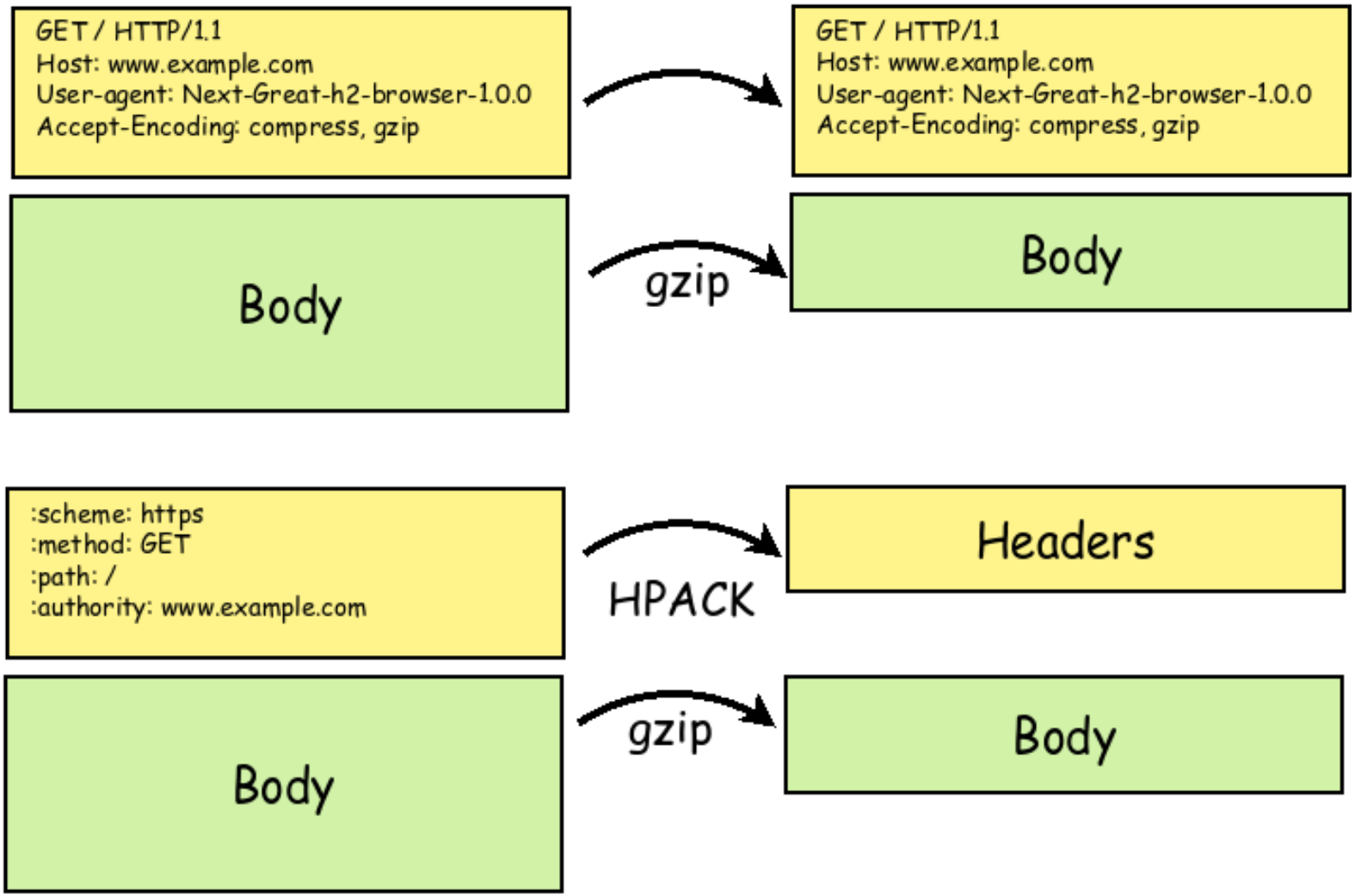
Estamos en:

1. HTTP/2 - Introducción
2. Funcionamiento
3. Stream y Frames
4. Control de Flujo
5. Server Push
6. Header Compression

HPACK

- En HTTP/1.1 es posible comprimir el payload.
- HTTP/2 permite hacerlo en cabeceras y payload.
- HPACK algoritmo de compresión, especialmente diseñado para compresión de las cabeceras.
- Utiliza dos mecanismos:
  - Huffman code para codificar las cabeceras cuando se envían.
  - Ambos extremos mantienen tablas indexadas de cabeceras previamente vistas:
    - Estática: provee un conjunto de cabeceras comunes definida en la especificación.
    - Dinámica: inicialmente vacía. Se va llenando dinámicamente.

HPACK (cont.)


GET / HTTP/1.1
Host: www.example.com
User-agent: Next-Great-h2-browser-1.0.0
Accept-Encoding: compress, gzip
GET / HTTP/1.1
Host: www.example.com
User-agent: Next-Great-h2-browser-1.0.0
Accept-Encoding: compress, gzip
Body
gzip
Body
:scheme: https
:method: GET
:path: /
:authority: www.example.com
HPACK
Headers
Body
gzip
Body

HPACK (cont.)

- Tabla estática contiene, predefinidas, hasta 61 cabeceras.
- Tabla dinámica permite agregar cabeceras a continuación de la estática.

Index	Header Name	Header Value
1	:authority	
2	:method	GET
3	:method	POST
4	:path	/
5	:path	/index.html
6	:scheme	http
7	:scheme	https
8	:status	200
.....		
61	www-authenticate	



Esta obra está sujeta a la licencia Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0) de Creative Commons.

Para detalle de esta licencia visite https://creativecommons.org/licenses/by-nc-sa/4.0/